# Exploring Neural Networks with TensorFlow Playground

*AAI101 — Module 4 Assignment*

Student: Patrick Hall

Tool: https://playground.tensorflow.org

## Assignment overview

For this lab I used TensorFlow Playground to see how different neural network settings change the decision boundary on the circle dataset. I kept settings consistent within each experiment and only changed one thing at a time. After training, I took a screenshot and wrote what I noticed and why I think it happened.

**Note on architecture:**

For Experiment 1 (activations) I used architecture 2 → 2 so I could compare ReLU, sigmoid, and tanh on the same small network. For Experiments 2–5 I used a stronger baseline of 4 → 2 (ReLU, LR 0.03, noise 0) unless that experiment was specifically changing architecture, learning rate, L2, or noise.

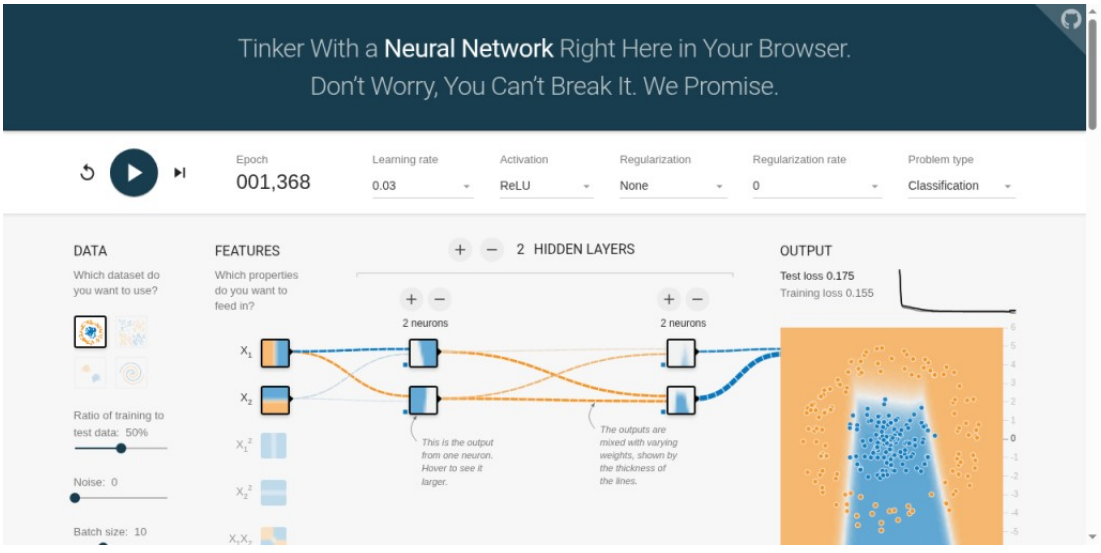
## Experiment 1 — Activation functions

I compared ReLU, sigmoid, and tanh with architecture fixed at 2,2 (and circle dataset, LR 0.03, noise 0). I wanted to see how activation changes the boundary when the network is small.

### 1a. ReLU (architecture 2,2)

Settings: activation = ReLU · architecture = 2,2 · LR 0.03 · noise 0 · circle dataset.

Results: Epoch 1,368 · Test loss 0.175 · Training loss 0.155



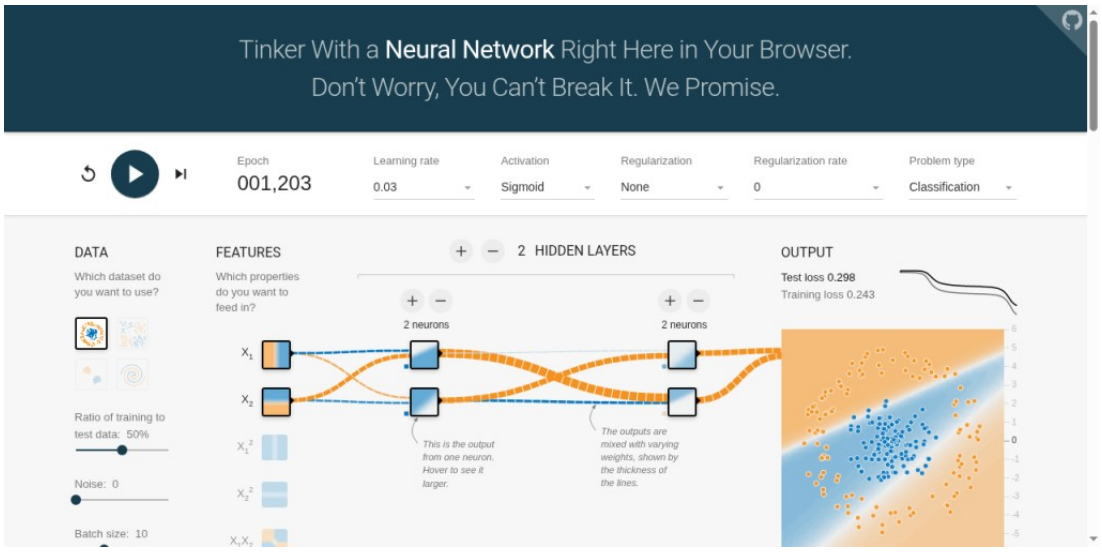

**Notes:**

With only 2 neurons in each hidden layer, ReLU never fully nailed the circle for me. After a long run the Output still looked more like a narrow blue band/wedge than a clean ring, and test loss stayed around 0.175. That makes sense — a circle needs a more flexible decision surface, and 2,2 just doesn't have much capacity. ReLU's straight half-plane style showed up in the angled pieces, but there weren't enough neurons to wrap all the way around the center.

## 1b. Sigmoid (architecture 2,2)

Settings: activation = Sigmoid · architecture = 2,2 · LR 0.03 · noise 0 · circle dataset.

Results: Epoch 1,203 · Test loss 0.298 · Training loss 0.243



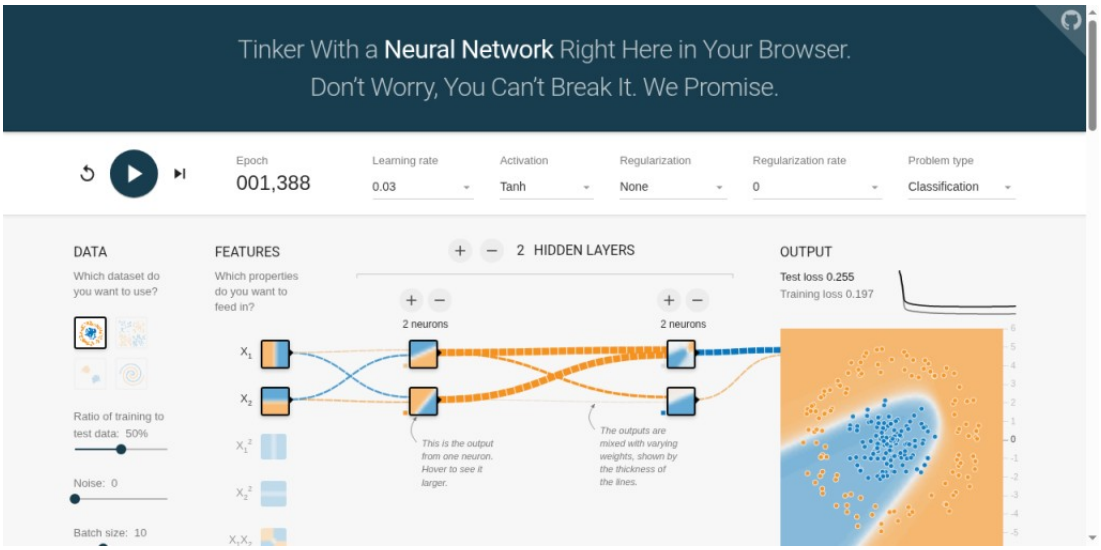

**Notes:**

Sigmoid struggled even more on the same 2,2 setup. The boundary looked mostly like a smooth diagonal separator, and test loss was about 0.298 — the worst of the three. I think sigmoid's smooth 0–1 squashing plus the tiny network made it hard to carve out an inner circle. So on this small architecture, changing activation didn't magically solve capacity — sigmoid still couldn't surround the blue points cleanly.

## 1c. Tanh (architecture 2,2)

Settings: activation = Tanh · architecture = 2,2 · LR 0.03 · noise 0 · circle dataset.

Results: Epoch 1,388 · Test loss 0.255 · Training loss 0.197




**Notes:**

Tanh did the best of the three on 2,2. The Output showed a more rounded blue area in the middle — closer to the inner circle — even though loss was still high (~0.255 test) and the fit wasn't perfect. That fits with tanh being smooth and zero-centered, so it can bend a little more naturally than ReLU's hard corners when capacity is limited. Still, none of the activations fully solved the circle at 2,2, which told me architecture size matters a lot here.

*What I took from Experiment 1: on a 2,2 network, activation changed the boundary style (angled vs diagonal vs more rounded), but none fully solved the circle. Tanh looked closest; sigmoid struggled most. The small architecture was the real limiter.*

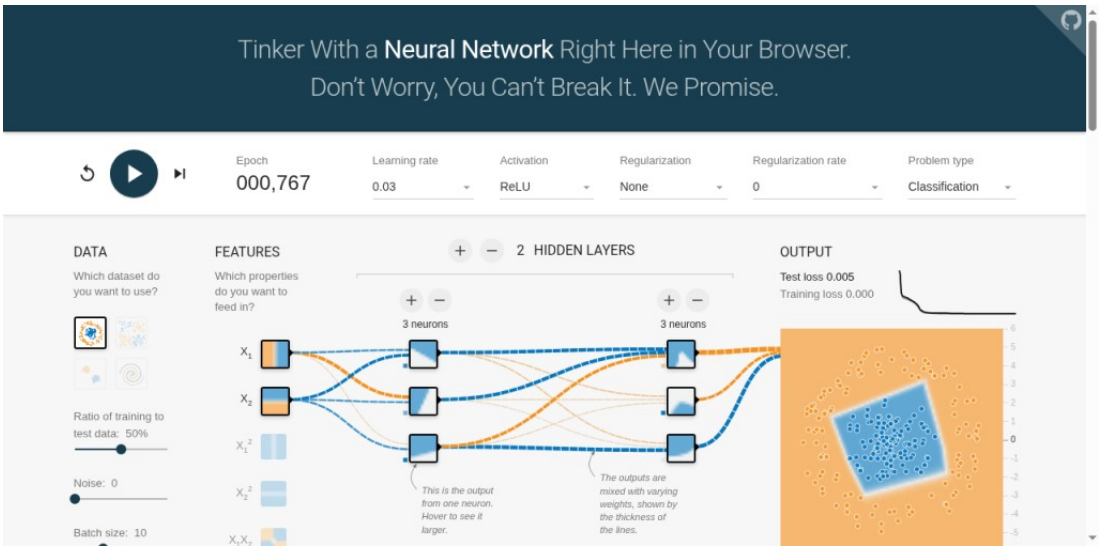
## Experiment 2 — Network architecture

Next I changed the network size/shape (baseline ReLU, LR 0.03, noise 0) to see how much capacity it needs to draw a circle.

### 2a. Architecture 3,3

Settings: network = 3,3; other settings at the Exp 2–5 baseline (ReLU, LR 0.03, noise 0).

Results: Epoch 767 · Test loss 0.005 · Training loss 0.000



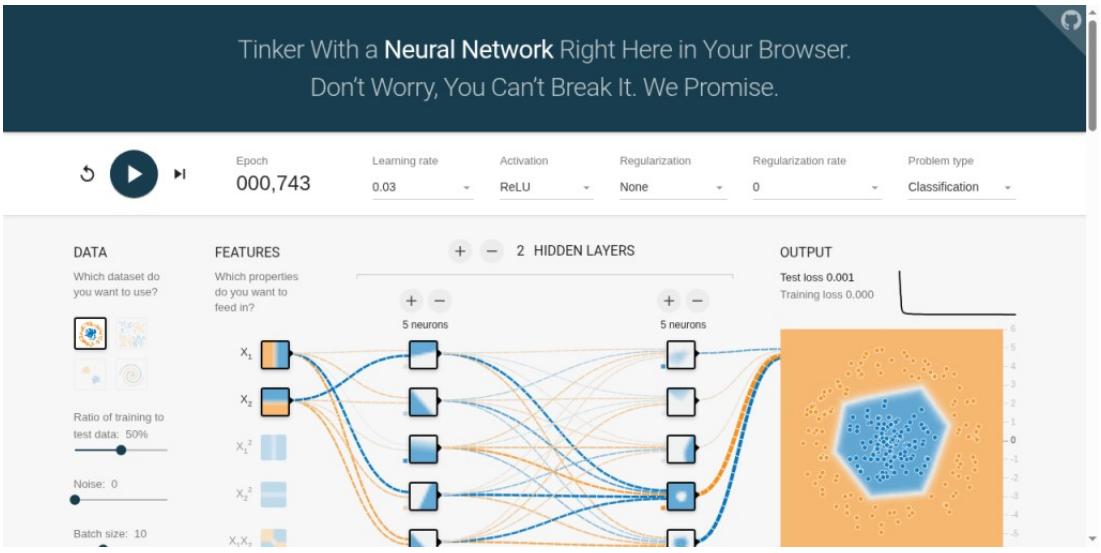

**Notes:**

With 3 neurons in each of two layers, it still made a clear blue center and orange outside. Loss got close to zero, so 3,3 was enough for this problem. Compared with my 2,2 activation runs, this was a big jump — more neurons finally let the model wrap a ring around the center. You don't need a huge network for a simple pattern, but you do need enough width.

## 2b. Architecture 5,5

Settings: network = 5,5; other settings at the Exp 2–5 baseline.

Results: Epoch 743 · Test loss 0.001 · Training loss 0.000



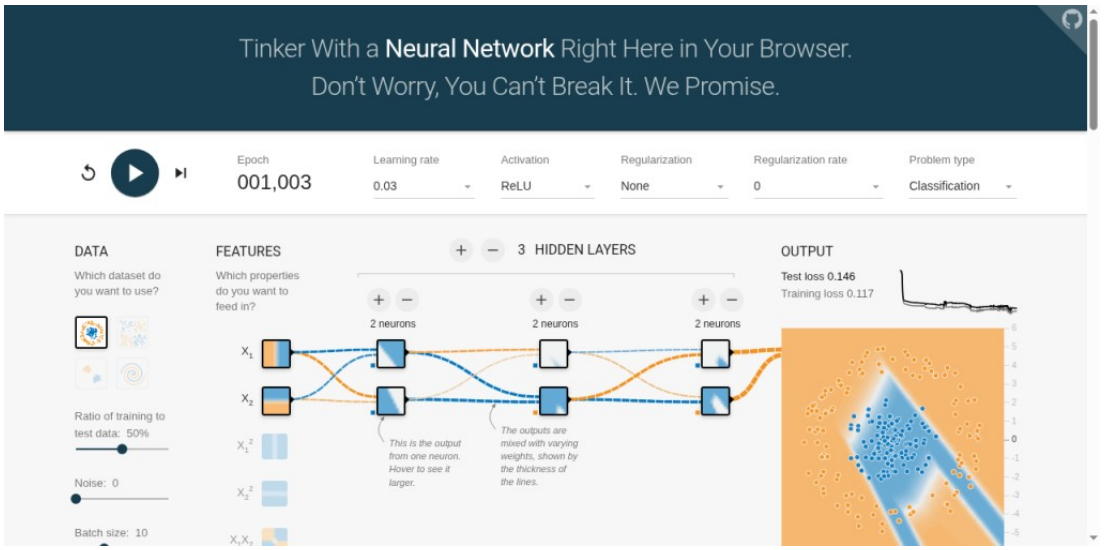

**Notes:**

Bumping it up to 5,5 made the boundary look cleaner and more compact, and test loss went to about 0.001. Having more neurons gives the model more pieces to shape the ring with, so it can fit the circle more tightly. The improvement over 3,3 was easy to see on the Output map.

### 2c. Architecture 2,2,2

Settings: network = 2,2,2 (three small layers); other settings at the Exp 2–5 baseline.

Results: Epoch 1,003 · Test loss 0.146 · Training loss 0.117




**Notes:**

This one lined up with what I saw in Experiment 1. A skinny deep net (2,2,2) never really got the circle — after a thousand epochs the boundary still looked broken/diagonal and loss stayed around 0.15. More layers didn't help when every layer was too thin. Wider nets like 3,3 and 5,5 beat this skinny deep one.

*What I took from Experiment 2: width mattered more than stacking tiny layers. 2,2 / 2,2,2 underfit; 3,3 and 5,5 did fine.*

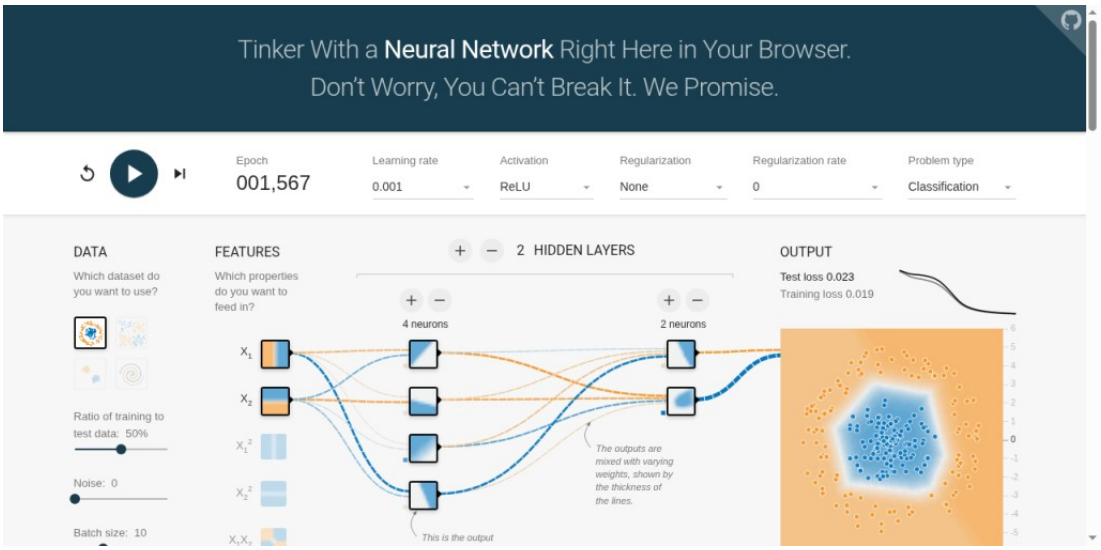
## Experiment 3 — Learning rate

I tried slow, medium, and fast learning rates with architecture 4,2 and ReLU to see how quickly the boundary forms.

### 3a. Learning rate 0.001

Settings: learning rate = 0.001; architecture 4,2; ReLU; noise 0.

Results: Epoch 1,567 · Test loss 0.023 · Training loss 0.019



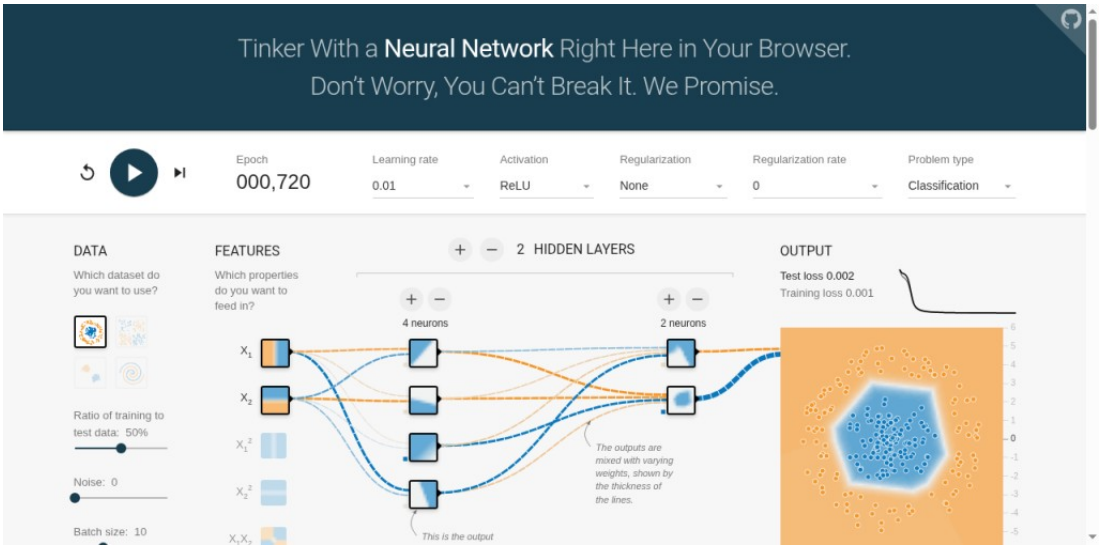

**Notes:**

With LR 0.001 it felt like watching the network crawl. I let it run a long time (1,500+ epochs) and it still wasn't as sharp as the other runs — test loss around 0.023. The circle was starting to show, just unfinished. A tiny learning rate only nudges the weights a little each step, so the boundary moves slowly.

## 3b. Learning rate 0.01

Settings: learning rate = 0.01; architecture 4,2; ReLU; noise 0.

Results: Epoch 720 · Test loss 0.002 · Training loss 0.001



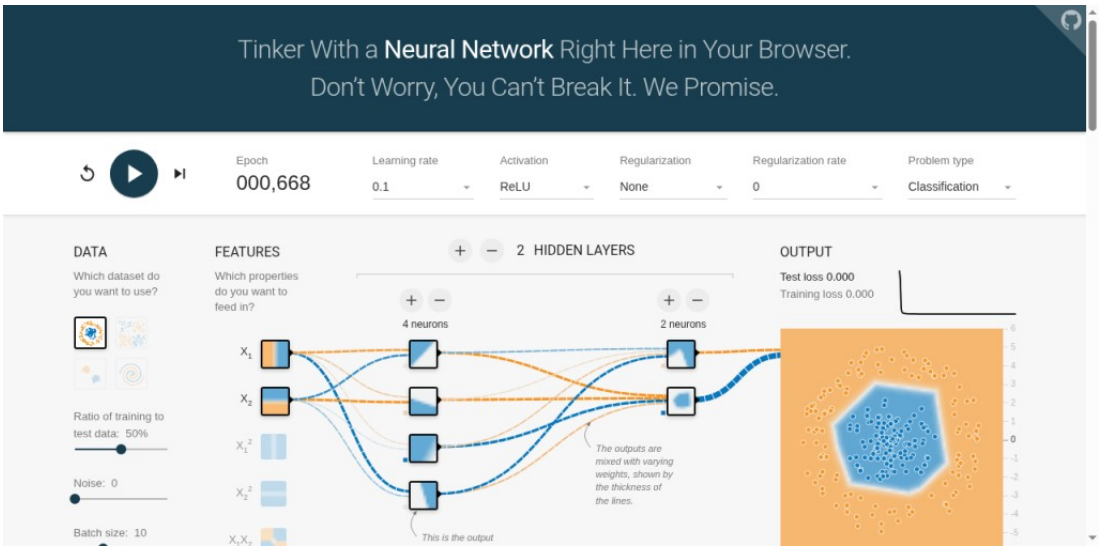

**Notes:**

0.01 felt like the sweet spot for this lab. By around 720 epochs I had a clean blue island and losses near zero. It trained way faster than 0.001 and didn't look unstable.

## 3c. Learning rate 0.1

Settings: learning rate = 0.1; architecture 4,2; ReLU; noise 0.

Results: Epoch 668 · Test loss 0.000 · Training loss 0.000




**Notes:**

I expected 0.1 to maybe bounce around, but on this simple circle with 4,2 it worked really well and hit basically zero loss with a crisp boundary. Bigger steps got it there faster. I know a high learning rate can fail on harder data, but here the Output plot looked great.

*What I took from Experiment 3: learning rate mostly changed how fast the boundary showed up. Too small = slow and incomplete; 0.01 and 0.1 both solved it on this dataset.*

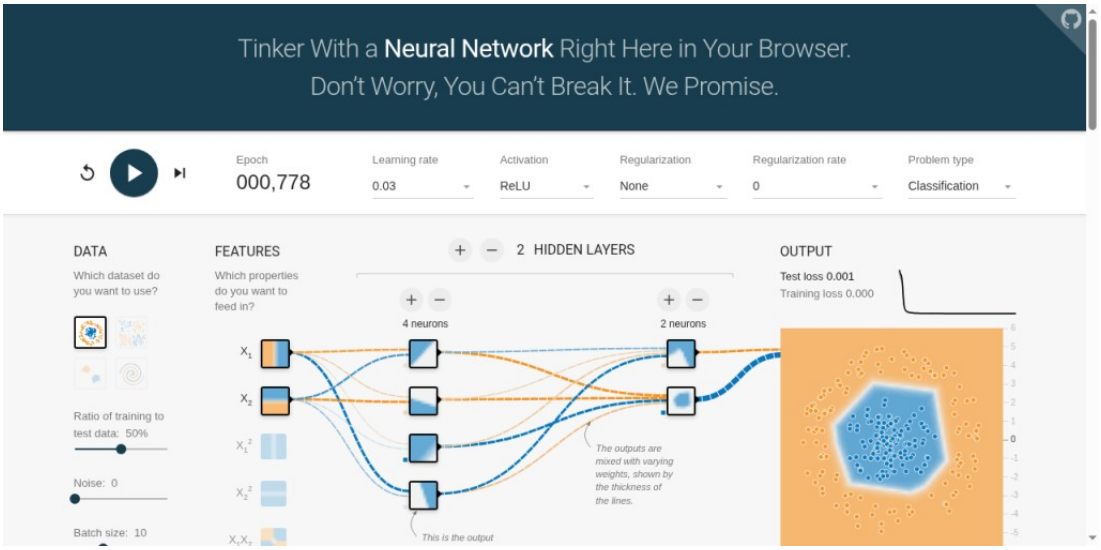
## Experiment 4 — L2 regularization

I compared no regularization vs light L2 vs strong L2 (architecture 4,2, ReLU, LR 0.03).

### 4a. No regularization

Settings: regularization off (rate 0); architecture 4,2; ReLU; LR 0.03; noise 0.

Results: Epoch 778 · Test loss 0.001 · Training loss 0.000



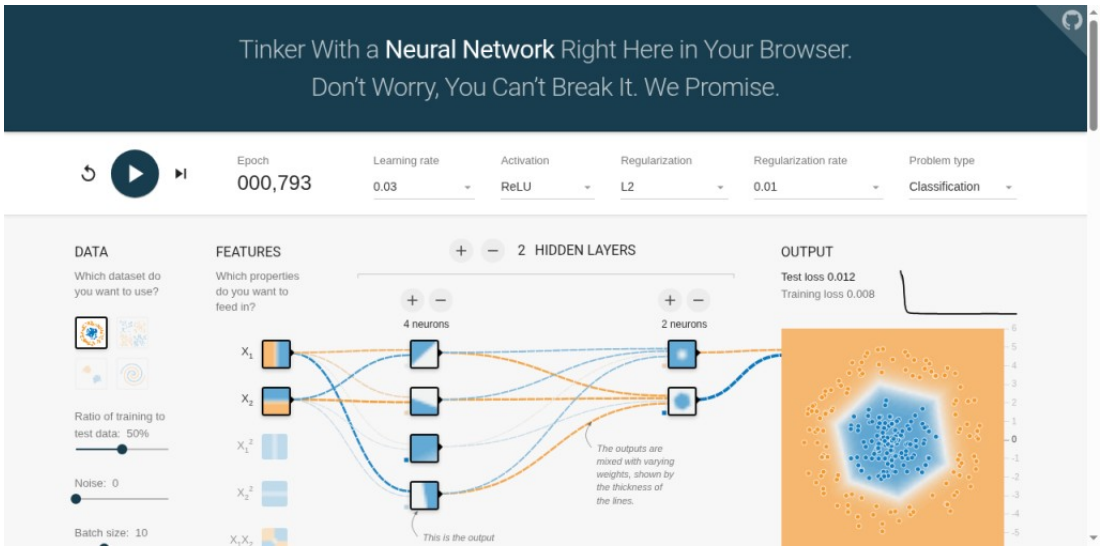

**Notes:**

With regularization off, this was a normal good run — clear circle, losses near zero. Nothing was holding the weights back, so the network could use whatever values it needed to separate the classes.

### 4b. L2 = 0.01

Settings: L2 regularization, rate 0.01; architecture 4,2; ReLU; LR 0.03; noise 0.

Results: Epoch 793 · Test loss 0.012 · Training loss 0.008



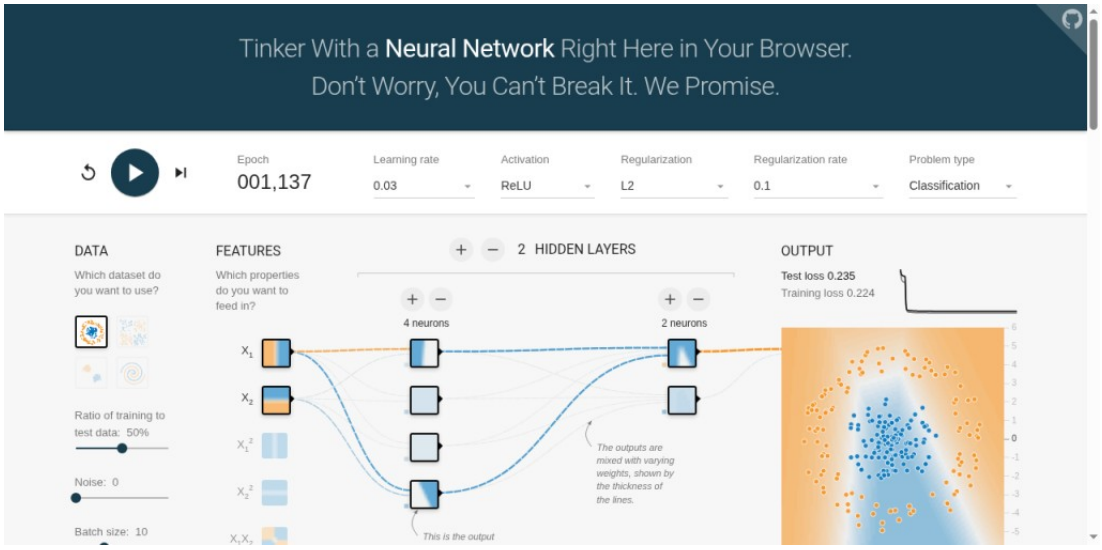

**Notes:**

Light L2 still found the circle, but the boundary looked a little softer and loss was a bit higher (~0.012 test). L2 gently pushes weights smaller, so the model stays a little simpler. It didn't break anything — just dialed the fit down a notch.

### 4c. L2 = 0.1

Settings: L2 regularization, rate 0.1; architecture 4,2; ReLU; LR 0.03; noise 0.

Results: Epoch 1,137 · Test loss 0.235 · Training loss 0.224




**Notes:**

Strong L2 basically wrecked the fit. Even after a long run, loss stayed high (~0.23) and the boundary turned into more of a diagonal smear instead of a ring. Both train and test loss were bad, so it wasn't overfitting — it just couldn't learn the circle. The penalty was so strong the weights couldn't get big enough to shape the ring.

*What I took from Experiment 4: a little L2 softened the boundary; a lot of L2 underfit hard. No regularization worked best on this noise-free circle.*

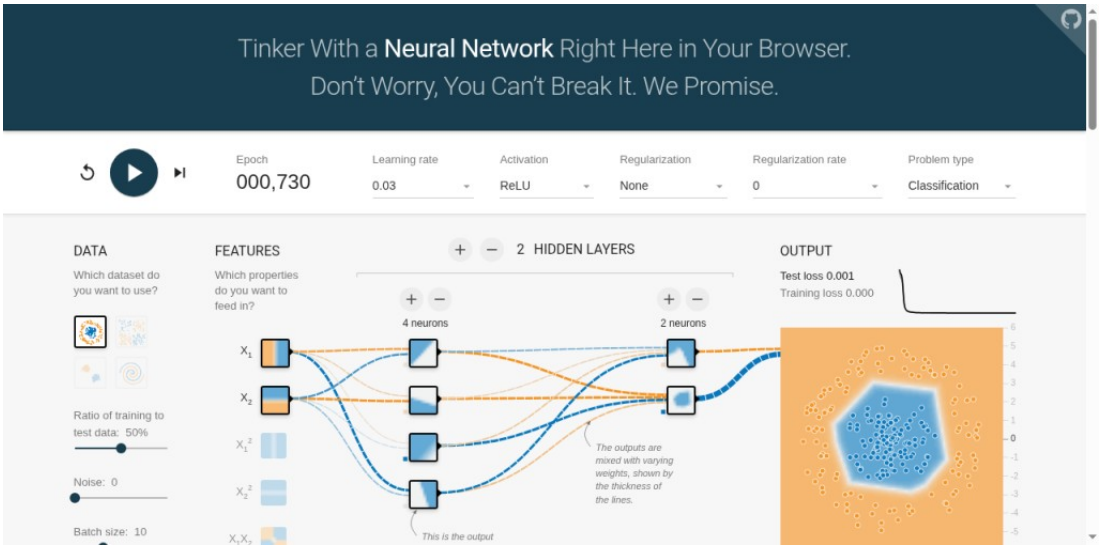
## Experiment 5 — Noise

Last I moved the noise slider (architecture 4,2, ReLU, LR 0.03) to see how messy data changes the boundary and the losses.

### 5a. Noise = 0

Settings: noise = 0; architecture 4,2; ReLU; LR 0.03.

Results: Epoch 730 · Test loss 0.001 · Training loss 0.000



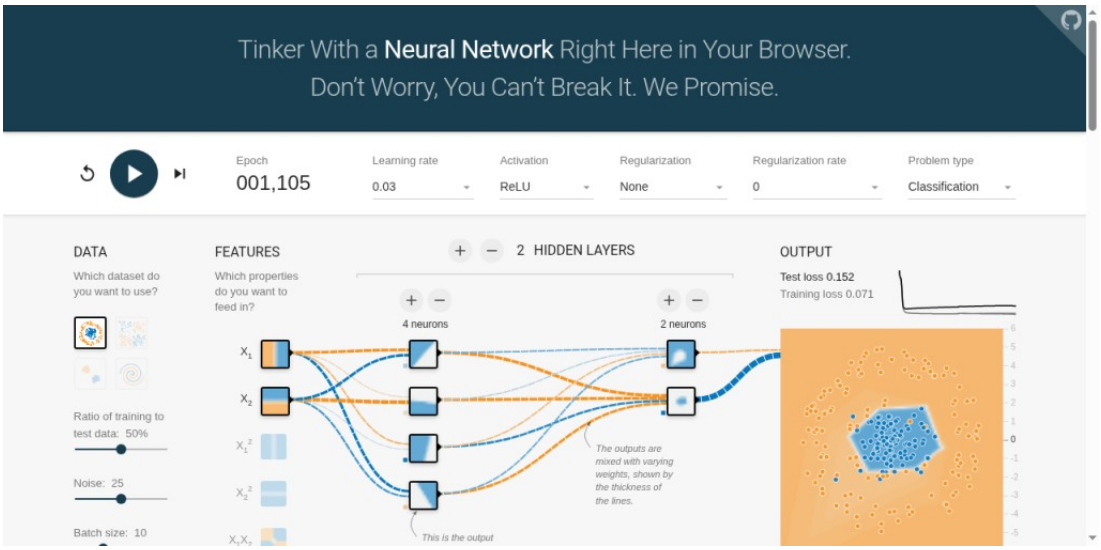

**Notes:**

With noise at 0, points stay where they belong, so the circle is obvious. Neat blue center, losses basically zero — what "easy data" looks like in Playground.

### 5b. Noise = 25

Settings: noise = 25; architecture 4,2; ReLU; LR 0.03.

Results: Epoch 1,105 · Test loss 0.152 · Training loss 0.071



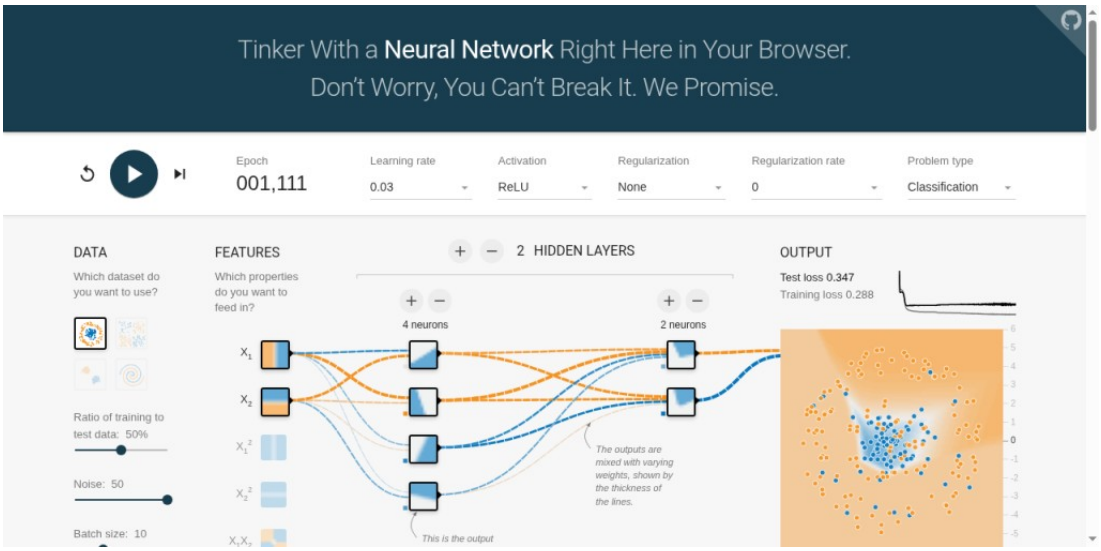

**Notes:**

At noise 25 the points start overlapping more, and the boundary gets blotchy instead of a clean ring. Training loss was about 0.07 but test loss was higher (~0.15). That gap stood out — the model fits some messy training points, then those rules don't work as well on test points. Noise hurt generalization, not just the picture.

## 5c. Noise = 50

Settings: noise = 50; architecture 4,2; ReLU; LR 0.03.

Results: Epoch 1,111 · Test loss 0.347 · Training loss 0.288




**Notes:**

Noise 50 made the dataset look chaotic. The classes are so mixed that the network never found a sharp circle, and both losses stayed high. If the data is too noisy, tweaking the model only helps so much — the boundary stayed fuzzy because the labels weren't consistent.

*What I took from Experiment 5: noise changed the boundary the most in a bad way. Clean data was easy; high noise made a fuzzy boundary and much higher test loss.*

## Overall reflection

After running all five experiments, the biggest thing I noticed is that the decision boundary shows what the network was able to learn under each setting. In Experiment 1 I kept architecture at 2,2 on purpose. That small net never fully solved the circle, but the activations still looked different — ReLU more angled/wedge-like, sigmoid more diagonal, tanh a bit more rounded in the center. That told me activation changes the style of the boundary, but capacity still matters. When I increased width in Experiment 2 (3,3 and 5,5), the circle finally came in clean, while skinny depth (2,2,2) underfit again. Learning rate mostly changed speed — 0.001 crawled, while 0.01 and 0.1 locked in a good boundary much faster once the network was big enough. L2 was about restraint: a little softened the fit, and too much stopped learning. Noise was the harshest change because it messes with the data itself, and that's when test loss jumped the most.

If I were tuning a real model after this lab, I'd make sure the network has enough width for the pattern, start with a mid learning rate, only add light regularization if I see overfitting, and care a lot about data quality. Bad noise and heavy L2 hurt more than picking sigmoid vs ReLU, and starting too small (like 2,2) can make any activation look bad. Overall this helped the idea of a decision boundary click for me — you can literally watch it change when you change one setting.

## Submission checklist

- ☐ Screenshot + notes for each experiment (after training)
- ☐ Notes explain why the boundary/loss changed, not only what I saw
- ☐ Overall reflection finished
- ☐ Saved/exported as one PDF for Canvas
- ☐ Optional Part B skipped unless I want extra practice